



FRONT-END

Buổi 7 : Javascript cơ bản (Tiết 1)



Nội dung

01

Khái niệm và giới thiệu

02

Variables (Biến)

03

Operators (Toán tử)

04

Data Types (Kiểu dữ liệu)

05

Một số hàm built-in

06

Typeof

07

Làm việc với String

08

Làm việc với Number

09

Làm việc với Array

01. Khái niệm và giới thiệu

1.1. Khái niệm

- Javascript (viết tắt là JS) là một ngôn ngữ lập trình kịch bản dựa vào các đối tượng có sẵn hoặc do lập trình viên tự định nghĩa.



HTML



HTML + CSS



HTML + CSS + JavaScript

01. Khái niệm và giới thiệu

1.2. Giới thiệu

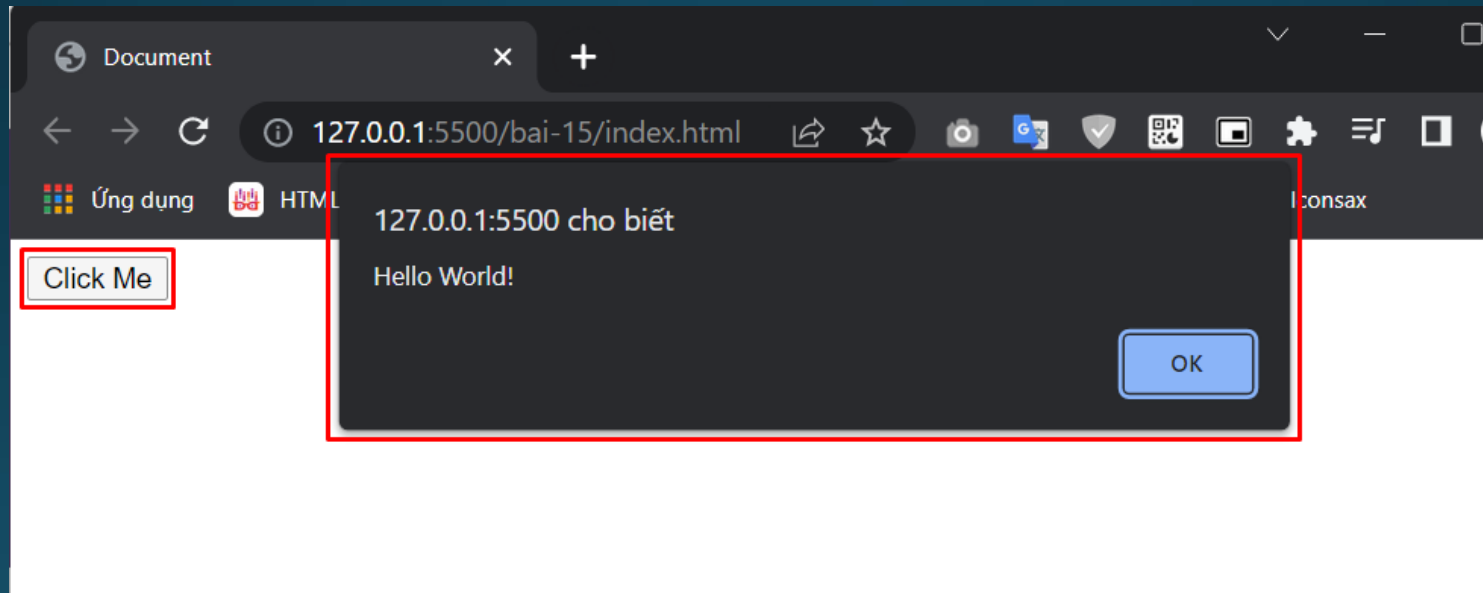
- Trước đây, Javascript được sử dụng chủ yếu để nâng cao sự tương tác của người dùng với trang web. Ví dụ làm các hiệu ứng đặc biệt như sliders, popup, hoặc xác thực dữ liệu các form (form validations) trước khi gửi dữ liệu lên server (máy chủ).
- Ngày nay, Javascript không chỉ giới hạn trong khuôn khổ xây dựng ứng dụng web, mà còn sử dụng rộng rãi trong phát triển ứng dụng, game trên điện thoại hay các ứng dụng dành cho server.
 - Web app: Reactjs, Vuejs, Angularjs...
 - Mobile app: React Native,...
 - Server app: Nodejs, Expressjs
 - Graphic: two.js (2D), three.js (3D)...

01. Khái niệm và giới thiệu

1.3. Viết chương trình Hello World!

```
<button onclick="alert('Hello  
World!')">
```

Click Me



01. Khái niệm và giới thiệu

1.4. Các cách sử dụng javascript

- **Cách 1: Inline** - viết trực tiếp trong thẻ HTML

```
<button onclick="alert('Hello World!')">  
    Click Me  
</button>
```

Cách 2: Internal - viết trong file HTML hiện tại

```
<script type="text/Javascript">  
var button = document.querySelector("button");  
    button.onclick = function() {  
        alert("Hello World!");  
    }  
</script>
```

Cách 3: External - viết ra một file js khác rồi nhúng vào trang HTML

02. Variables (Biến)

2.1. Khái niệm

- **Biến (variable)** là nơi để bạn có thể lưu trữ thông tin.

2.2. Khai báo biến bằng từ khóa var (variable - biến)

- Cú pháp: **var tenBien = giaTri;**
- Trong đó:
 - **tenBien**: Là tên của biến các bạn muốn đặt.
 - **giaTri**: Là giá trị của biến, có thể là số, chuỗi, mảng, object.
- Ví dụ:

```
var a = 15;  
var b = 20;  
var c = a + b; => Kết quả c = 35.
```
- **Lưu ý**: Tên biến có phân biệt chữ hoa, chữ thường.

02. Variables (Biến)

2.3. Cách đặt tên cho biến

- Quy tắc đặt tên biến:
 - Bắt đầu phải là chữ cái hoặc ký tự '_'
 - Không được bắt đầu bằng số (o → 9).
 - Không được bắt đầu bằng ký tự đặc biệt (ví dụ: #, %, ^, -)

Đặt tên biến ĐÚNG:

- `var tenbien = 10;`
- `var _tenbien = 10;`
- `var TenBien = 10; // PascalCase`
- `var ten_bien = 10; // snake_case`
- `var tenBien = 10; // camelCase`
- `var tenbien123 = 10;`

Đặt tên biến SAI:

- `var 123tenbien = 10; // Do số đứng đầu`
- `var -tenbien = 10; // Do có ký tự -`



03. Operators (Toán tử)

3.1. Arithmetic (Toán tử số học)

- Toán tử số học là toán tử dùng để thực hiện các phép toán số học.
- Danh sách các toán tử số học:

Toán tử	Mô tả
+	Phép cộng (Nếu là chuỗi thì nó sẽ thực hiện thao tác nối chuỗi)
-	Phép trừ
*	Phép nhân
**	Phép lũy thừa
/	Phép chia
%	Phép chia lấy phần dư, nghĩa là khi chia hai số với nhau và kết quả nó sẽ lấy phần dư của phép toán.
++	Phép tăng giá trị hiện tại lên 1 đơn vị.
--	Phép giảm giá trị hiện tại xuống 1 đơn vị.

03. Operators (Toán tử)

3.1. Arithmetic (Toán tử số học)

- Toán tử ++
 - Phép **tăng** giá trị hiện tại lên **1 đơn vị**
 - Hai cách sử dụng:
 - Đặt trước biến (prefix - tiền tố): nó sẽ tăng trước khi lấy giá trị
 - Đặt sau biến (postfix - hậu tố): nó sẽ lấy giá trị rồi mới tăng lên
- Toán tử --
 - Phép **giảm** giá trị hiện tại xuống **1 đơn vị**
 - Hai cách sử dụng:
 - Đặt trước biến (prefix - tiền tố): nó sẽ giảm trước khi lấy giá trị
 - Đặt sau biến (postfix - hậu tố): nó sẽ lấy giá trị rồi mới giảm xuống

03. Operators (Toán tử)

3.2. Assignment (Toán tử gán)

- Toán tử gán dùng để gán giá trị cho một biến.
- Danh sách các toán tử gán:

Toán tử	Ví dụ	Mô tả
=	a = b	Gán giá trị của biến a bằng giá trị của biến b
+=	a += b	Tương đương với $a = a + b$
-=	a -= b	Tương đương với $a = a - b$
*=	a *= b	Tương đương với $a = a * b$
/=	a /= b	Tương đương với $a = a / b$
%=	a %= b	Tương đương với $x = x \% y$

03. Operators (Toán tử)

3.3. Comparison (Toán tử so sánh)

- **Toán tử so sánh** là toán tử hai ngôi dùng để so sánh giá trị của hai toán hạng với nhau.
- **Danh sách các toán tử so sánh:**

Toán tử	Ví dụ	Mô tả
>	$a > b$	Trả về true nếu a lớn hơn b. Trả về false nếu b lớn hơn hoặc bằng a.
<	$a < b$	Trả về true nếu a nhỏ hơn b. Trả về false nếu b nhỏ hơn hoặc bằng a.
>=	$a >= b$	Trả về true nếu a lớn hơn hoặc bằng b. Trả về false nếu a nhỏ hơn b.
<=	$a <= b$	Trả về true nếu a nhỏ hơn hoặc bằng b. Trả về false nếu a lớn hơn b.

03. Operators (Toán tử)

3.3. Comparison (Toán tử so sánh)

- **Toán tử so sánh** là toán tử hai ngôi dùng để so sánh giá trị của hai toán hạng với nhau.
- **Danh sách các toán tử so sánh:**

Toán tử	Ví dụ	Mô tả
==	a == b	Trả về true nếu a bằng b. Trả về false nếu a khác b. Không nghiêm ngặt.
===	a === b	Trả về true nếu giá trị của a bằng giá trị của b. Với a, b phải cùng kiểu dữ liệu. Nghiêm ngặt.
!=	a != b	Trả về true nếu a khác b. Trả về false nếu a bằng b. Không nghiêm ngặt.
!==	a !== b	Trả về true nếu giá trị của a khác giá trị của b Với a, b phải khác kiểu dữ liệu. Nghiêm ngặt.

03. Operators (Toán tử)

3.4. Logical (Toán tử logic)

- **Toán tử logic** là toán tử kết nối hai hay nhiều biểu thức, dùng để kiểm tra mối quan hệ logic giữa các biểu thức. Kết quả cuối cùng phụ thuộc vào giá trị của từng biểu thức và loại toán tử logic.
- **Danh sách các toán tử logic:**

Toán tử	Mô tả
&&	AND: trả về kết quả là true khi cả hai toán hạng đều true.
 	OR: trả về kết quả là true khi cả hai hoặc một trong hai toán hạng là true.
!	NOT: Chuyển đổi giá trị của toán hạng từ true sang false hoặc từ false sang true.

04. Data Types (Kiểu dữ liệu)

4.1. Kiểu dữ liệu nguyên thủy (Primitive Data)

- **Khái niệm:** Kiểu dữ liệu nguyên thủy là kiểu dữ liệu mà giá trị không thể thay đổi được.
- **Có 6 kiểu:** Number, String, Boolean, Undefined, Null, Symbol
- **Kiểu Number**
 - Kiểu dữ liệu number là kiểu dữ liệu **dạng số** (tương tự trong toán học).
 - JavaScript có hai loại số là: **số nguyên** và **số thực**.
 - Ngoài những loại số trên, JavaScript còn có **3 số đặc biệt** là:
 - **Infinity**: là số dương vô cùng.
 - **-Infinity**: là số âm vô cùng.
 - **NaN**: là viết tắt của Not a Number, được sử dụng để đại diện cho những trường hợp tính toán sai hoặc kết quả của một phép tính không xác định.
- **Ví dụ:**
 - `var a = 10; // Số nguyên`
 - `var b = 10.5; // Số thực`
 - `var c = Infinity; // Số dương vô cùng`
 - `var d = -Infinity; // Số âm vô cùng`
 - `var e = 10/"a"; // NaN`

04. Data Types (Kiểu dữ liệu)

4.1. Kiểu dữ liệu nguyên thủy (Primitive Data)

- **Kiểu String**
 - String là kiểu dữ liệu dùng để **biểu diễn chữ, văn bản, đoạn văn bản,...**
 - Có **3 cách** để biểu diễn string trong JavaScript:
 - Dùng dấu nháy đơn: `' '`
 - Dùng dấu nháy kép: `" "`
 - Dùng dấu "backtick": `` ``
 - Ví dụ:
 - `const msg1 = 'Đây là string dùng dấu nháy đơn';`
 - `const msg2 = "Đây là string dùng dấu nháy kép";`
 - `const msg3 = `Đây là string dùng dấu backtick`; // template string trong ES6`

04. Data Types (Kiểu dữ liệu)

4.1. Kiểu dữ liệu nguyên thủy (Primitive Data)

- **Kiểu Boolean**
 - Boolean là kiểu dữ liệu **logic** chỉ bao gồm 2 giá trị là **true** (đúng, chính xác) và **false** (sai, không chính xác)
 - Ví dụ:
 - `var isDarkMode = true; // Chế độ dark mode`
 - `var isDarkMode = false; // Chế độ light mode`

04. Data Types (Kiểu dữ liệu)

4.1. Kiểu dữ liệu nguyên thủy (Primitive Data)

- **Kiểu Undefined**
 - Undefined là một kiểu dữ liệu mà khi khai báo ra một biến và không gán giá trị thì kết quả trả về là undefined.
 - Ví dụ:

```
var a;
```

```
console.log(a) // undefined
```

04. Data Types (Kiểu dữ liệu)

4.1. Kiểu dữ liệu nguyên thủy (Primitive Data)

- **Kiểu Null**

- Kiểu dữ liệu null là một kiểu dữ liệu đặc biệt, chỉ bao gồm một **giá trị là null** (không biết giá trị, không có giá trị).

- Ví dụ:

```
var a = null;
```

```
console.log(a) // null
```

- **Khác nhau cơ bản giữa undefined và null**

- Kiểu dữ liệu **null** là kiểu dữ liệu **được gán cho biến**, thường được hiểu là **không biết** (không có).
- Kiểu dữ liệu **undefined** là giá trị **mặc định** của biến sau khi khai báo mà **không gán giá trị cho biến**.

04. Data Types (Kiểu dữ liệu)

4.1. Kiểu dữ liệu nguyên thủy (Primitive Data)

- **Kiểu Symbol**
 - Symbol là một kiểu dữ liệu nguyên thủy dùng để tạo ra các **giá trị duy nhất** (unique value) và **bất biến** (immutable).
 - Symbol thường được dùng làm key cho kiểu dữ liệu object.
 - Ví dụ: Tự nghiên cứu thêm

04. Data Types (Kiểu dữ liệu)

4.2. Dữ liệu phức tạp (Complex Data) (Các buổi sau học chi tiết)

- Kiểu Function

- Function (hàm) trong JavaScript là một chương trình con giúp thực thi một công việc cụ thể.
- Cú pháp:

```
function tenHam(thamSo1, thamSo2,...) {  
    // Code  
}
```

- Ví dụ:

```
function sum(a, b) {  
    alert(a + b);  
}  
sum(10, 20);
```

04. Data Types (Kiểu dữ liệu)

4.2. Dữ liệu phức tạp (Complex Data) (Các buổi sau học chi tiết)

- **Kiểu Object**
 - Object là kiểu dữ liệu tham chiếu. Có thể hiểu object là một tập hợp gồm các cặp key - value (khóa - giá trị).
 - Kiểu dữ liệu của key có thể là string hoặc symbol.
 - Value ứng với key có thể là bất kỳ kiểu dữ liệu nào.
 - Có 2 loại là Object và Array.
 - Ví dụ về Object

```
var infoUser = {  
  name: "Le Van A",  
  age: 18,  
  email: "levana@gmail.com",  
  phone: "0987654321",  
}  
console.log(infoUser.age);
```

04. Data Types (Kiểu dữ liệu)

4.2. Dữ liệu phức tạp (Complex Data) (Các buổi sau học chi tiết)

- Kiểu Object

- Object là kiểu dữ liệu tham chiếu. Có thể hiểu object là một tập hợp gồm các cặp key - value (khóa - giá trị).
- Kiểu dữ liệu của key có thể là string hoặc symbol.
- Value ứng với key có thể là bất kỳ kiểu dữ liệu nào.
- Có 2 loại là Object và Array.
- Ví dụ về Array

```
var listUser = [  
    "Le Van A",  
    "Nguyen Thi B",  
    "Do Van C"  
]  
console.log(listUser);  
console.log(listUser[0]);
```

05. Một số hàm built-in

5.1. Khái niệm

- Hàm built-in là những hàm được javascript định nghĩa sẵn, chúng ra chỉ việc sử dụng.

5.2. Một số hàm built-in

- Hàm **alert()**
 - Hàm **alert()** có nhiệm vụ in một thông báo popup
 - Nó có một tham số truyền vào và tham số này chính là nội dung ta muốn thông báo với người dùng.
 - Cú pháp: **alert(message);**
 - Ví dụ: `alert("Xin chào!");`

05. Một số hàm built-in

5.2. Một số hàm built-in

- Hàm **confirm()**
 - Hàm **confirm()** cũng sẽ xuất hiện một **thông báo popup** nhưng nó **có thêm hai sự lựa chọn** là **Yes** và **No**.
 - Nếu người dùng **chọn Yes** thì nó trả về **TRUE** và ngược lại nếu **chọn NO** thì nó sẽ trả về **FALSE**.
 - Nó cũng có **một tham số truyền vào** và tham số này chính là nội dung thông báo.
 - Cú pháp: **confirm(message);**
 - Ví dụ:

```
var result = confirm("Bạn đã trên 18 tuổi?");  
alert(result);
```

05. Một số hàm built-in

5.2. Một số hàm built-in

- Hàm **prompt()**
 - Hàm **prompt()** dùng để lấy thông tin từ người dùng
 - Gồm có hai tham số truyền vào là nội dung thông báo và giá trị ban đầu.
 - Cú pháp: **prompt(title, defaultValue);**
 - Trong đó:
 - **title**: nội dung chữ hiển thị tới người dùng.
 - **defaultValue**: giá trị mặc định cho ô nhập và đây là giá trị không bắt buộc.
 - Ví dụ:

```
const name = prompt("Nhập vào tên của bạn:", "Le Van A");  
alert(name);
```

05. Một số hàm built-in

5.2. Một số hàm built-in

- Hàm **console**
 - Dùng để **in ra nội dung ở tab console** trên dev tools.
 - Có 3 hàm hay sử dụng là:
 - `console.log()`: in ra **thông báo**
 - `console.warn()`: in ra **cảnh báo**
 - `console.error()`: in ra **lỗi**
 - Nghiên cứu thêm tại: <https://developer.mozilla.org/en-US/docs/Web/API/console>

05. Một số hàm built-in

5.2. Một số hàm built-in

- Hàm **setTimeout**
 - Code sẽ **chạy 1 lần duy nhất sau một khoảng thời gian**.

- Cú pháp:

```
setTimeout(function() {  
    // Code  
}, time)
```

- Ví dụ:

```
setTimeout(function(){  
    console.log("Xin chào!");  
}, 5000)
```

05. Một số hàm built-in

5.2. Một số hàm built-in

- Hàm **setInterval**

- Code sẽ **chạy lặp lại** sau một khoảng thời gian.

- Cú pháp:

```
setInterval(function(){
```

```
    // Code
```

```
}, time)
```

- Ví dụ:

```
setInterval(function(){  
    console.log("Xin chào!");  
}, 5000)
```

o6. Typeof

- Các toán tử **typeof** cho phép **trả về kiểu** của một **biến**.
- Nó hữu ích khi chúng ta muốn xử lý các giá trị của các kiểu khác nhau hoặc chỉ muốn kiểm tra nhanh.
- Cú pháp: **typeof x** (với **x** là một biến hoặc biểu thức)
- Ví dụ:

```
var a = 10;
```

```
var b = "Nội dung...";
```

```
var c;
```

```
var d = null;
```

```
var e = true;
```

```
console.log(typeof a); // number
```

```
console.log(typeof b); // string
```

```
console.log(typeof (a + b)); // string
```

```
console.log(typeof c); // undefined
```

```
console.log(typeof d); // object
```

```
console.log(typeof e); // boolean
```

```
console.log(typeof a == "number"); // true
```

07. Làm việc với String

7.1. Length

- Length trả về **độ dài** của chuỗi.
- Ví dụ:

```
var fullName = `Le Van A`;
```

```
console.log(fullName.length);
```

07. Làm việc với String

7.2. indexOf()

- **Tìm vị trí đầu tiên** của một chuỗi nằm trong một chuỗi.
- Nếu **không tìm thấy** sẽ trả về **-1**.
- Có phân biệt chữ hoa, chữ thường.
- Cú pháp: **string.indexOf(searchvalue, start)**
- Trong đó:
 - **searchvalue**: Giá trị cần tìm kiếm (bắt buộc).
 - **start**: Vị trí bắt đầu tìm kiếm (tính từ trái qua phải) và sẽ tìm kiếm xuôi tiếp (mặc định bắt đầu từ 0).
- Ví dụ:

```
var myString = `Xin chào! Tôi tên Nam. Tôi năm nay 18 tuổi.`;  
console.log(myString.indexOf("Tôi")); // Trả về 10  
console.log(myString.indexOf("Hải")); // Trả về -1  
console.log(myString.indexOf("Nam")); // Trả về 18  
console.log(myString.indexOf("nam")); // Trả về -1  
console.log(myString.indexOf("Tôi", 11)); // Trả về 23
```


07. Làm việc với String

7.3. lastIndexOf()

- **Tìm vị trí cuối cùng** của một chuỗi nằm trong một chuỗi.
- Nếu **không tìm thấy** sẽ trả về **-1**.
- Có phân biệt chữ hoa, chữ thường.
- Cú pháp: **string.lastIndexOf(searchvalue, start)**
- Trong đó:
 - **searchvalue**: Giá trị cần tìm kiếm (bắt buộc).
 - **start**: Vị trí bắt đầu tìm kiếm (tính từ trái qua phải) và sẽ tìm kiếm ngược lại (mặc định bắt đầu từ 0).
- Ví dụ:

```
var myString = `Xin chào! Tôi tên Nam. Tôi năm nay 18 tuổi.`;  
console.log(myString.lastIndexOf("Tôi", 24)); // Trả về 23  
console.log(myString.lastIndexOf("Tôi", 22)); // Trả về 10  
console.log(myString.lastIndexOf("Tôi", 10)); // Trả về 10  
console.log(myString.lastIndexOf("Tôi", 9)); // Trả về -1
```

07. Làm việc với String

7.4. slice()

- Dùng để cắt một chuỗi và trả về một chuỗi mới.
- Chuỗi ban đầu không thay đổi.
- Cú pháp: **string.slice(start, end)**
- Trong đó:
 - **start**: Vị trí bắt đầu cắt (bắt buộc).
 - **end**: Vị trí kết thúc cắt (không bắt buộc).

- Ví dụ:

```
var myString = `Xin chào! Tôi tên Nam. Tôi năm nay 18 tuổi.`;  
console.log(myString.slice(10, 21)); // Tôi tên Nam  
console.log(myString.slice(0)); // Lấy toàn bộ chuỗi  
console.log(myString.slice(0, 1)); // X (Lấy ký tự đầu tiên của chuỗi)  
console.log(myString.slice(-1)); // . (Lấy ký tự cuối cùng của chuỗi)  
console.log(myString); // Vẫn giữ nguyên chuỗi gốc
```

07. Làm việc với String

7.5. replace()

- Dùng để **thay thế** 1 chuỗi thành 1 chuỗi mới.
- Nhưng chỉ **thay thế chuỗi đầu tiên** mà nó **tìm thấy**.
- Muốn thay thế được tất cả các từ tìm thấy thì phải dùng Regular Expression (Biểu thức chính quy).
- Cú pháp: **string.replace(searchValue, newValue)**
- Trong đó:
 - **searchValue**: Từ khóa cần thay thế (bắt buộc).
 - **newValue**: Từ khóa mới để thay thế (bắt buộc).
- Ví dụ:

```
var myString = `Xin chào! Tôi tên Nam. Tôi năm nay 18 tuổi.`;  
console.log(myString.replace("Tôi", "Mình"));  
console.log(myString.replace(/Tôi/g, "Mình"));
```

07. Làm việc với String

7.6. toUpperCase() và toLowerCase()

- toUpperCase() dùng để viết hoa tất cả các chữ cái.
- toLowerCase() dùng để viết thường tất cả các chữ cái.
- Ví dụ:

```
var myString = `Xin chào! Tôi tên Nam. Tôi năm nay 18 tuổi.`;  
console.log(myString.toUpperCase());  
console.log(myString.toLowerCase());
```

07. Làm việc với String

7.7. trim()

- Dùng để **bỏ đi khoảng trắng ở 2 đầu**, các khoảng trắng giữa các ký tự không bị loại bỏ.
- Ví dụ:

```
var myString = `  Xin chào! Tôi tên Nam.  Tôi năm   nay 18 tuổi.  `;  
console.log(myString);  
console.log(myString.trim());
```

7.8. charAt()

- Dùng để lấy ký tự thông qua index (với index là vị trí của ký tự cần lấy).
- Cú pháp: **string.charAt(index)**
- Ví dụ:

```
var myString = `Đăng Phương Nam`;  
console.log(myString.charAt()); // Lấy ký tự đầu tiên  
console.log(myString.charAt(0)); // Lấy ký tự đầu tiên  
console.log(myString.charAt(1)); // Lấy ký tự thứ 2  
console.log(myString.charAt(myString.length - 1)); // Lấy ký tự cuối cùng
```

07. Làm việc với String

7.9. split()

- Chuyển một chuỗi thành một array, nhưng cần tìm ra điểm chung của chuỗi đó.
- Ví dụ:

```
var myString = `HTML5, CSS3, Javascript`;
console.log(myString.split()); // Cả string là 1 phần tử
console.log(myString.split("")); // Mỗi ký tự là 1 phần tử
console.log(myString.split(", ")); // Mỗi từ là 1 phần tử
console.log(myString.split(", ", 2)); // Mỗi từ là 1 phần tử, lấy tối đa 2 phần tử
```

o8. Làm việc với Number

8.1. isNaN()

- Dùng để **kiểm tra** một biến có phải là **NaN** không.
- (NaN: là viết tắt của Not a Number, được sử dụng để đại diện cho những trường hợp tính toán sai hoặc kết quả của một phép tính không xác định)
- Cú pháp: **isNaN(tenBien)**
- Ví dụ:

```
var a = 10;  
var b = "Test";  
var result = a / b;  
console.log(result); // Trả về NaN  
console.log(isNaN(result)); // Trả về true
```

o8. Làm việc với Number

8.2. toString()

- Chuyển kiểu số thành kiểu string.
- Cú pháp: **tenBien.toString()**
- Ví dụ:

```
var a = 10;
var b = a.toString();
var c = (10).toString();
console.log(a); // Trả về số 10
console.log(typeof a); // Trả về kiểu number
console.log(b); // Trả về chuỗi 10
console.log(typeof b); // Trả về kiểu string
console.log(c); // Trả về chuỗi 10
console.log(typeof c); // Trả về kiểu string
```


o8. Làm việc với Number

8.3. toFixed()

- Dùng để **làm tròn số thập phân**.
- Nếu số ngay sau số cần làm tròn mà **≥ 5** thì sẽ **làm tròn lên**, nếu **< 5** sẽ **làm tròn xuống**.
- Cú pháp: **toFixed(digits)**
- Trong đó: **digits** là số chữ số sau dấu thập phân ($0 \leq \text{digits} \leq 100$). Không điền mặc định là 0.
- Ví dụ:

```
var a = 12.3456;  
console.log(a.toFixed()); // Trả về 12  
console.log(a.toFixed(0)); // Trả về 12  
console.log(a.toFixed(1)); // Trả về 12.3  
console.log(a.toFixed(2)); // Trả về 12.35  
console.log(a.toFixed(3)); // Trả về 12.346
```

09. Làm việc với Array

9.1. toString()

- Chuyển array thành string. Tự động thêm dấu phẩy ngăn cách.
- Cú pháp: **array.toString()**
- Ví dụ:

```
var list = ["HTML5", "CSS3", "Javascript"];  
console.log(list.toString()); // Trả về: "HTML5,CSS3,Javascript"
```

09. Làm việc với Array

9.2. join()

- Chuyển array thành string. Thêm dấu bất kỳ để ngăn cách giữa các phần tử.
- Cú pháp: **array.join(separator)**
- Trong đó: **separator** (dấu phân cách) là dấu ngăn cách để thêm vào string. Mặc định là dấu phẩy.
- Ví dụ:

```
var list = ["HTML5", "CSS3", "Javascript"];
console.log(list.join()); // Trả về: "HTML5,CSS3,Javascript"
console.log(list.join(", ")); // Trả về: "HTML5,CSS3,Javascript"
console.log(list.join("")); // Trả về: "HTML5CSS3Javascript"
console.log(list.join(" ", " ")); // Trả về: "HTML5, CSS3, Javascript"
console.log(list.join(" - ")); // Trả về: "HTML5 - CSS3 - Javascript"
```

09. Làm việc với Array

9.3. pop()

- Dùng để **xóa phần tử cuối mảng** và **trả về phần tử cuối mảng** đó.
- Cú pháp: **array.pop()**
- Ví dụ:

```
var list = ["HTML5", "CSS3", "Javascript"];  
console.log(list.pop()); // Trả về: "Javascript"  
console.log(list); // Trả về: ["HTML5", "CSS3"]
```

09. Làm việc với Array

9.4. push()

- **Thêm** một hoặc nhiều **phần tử vào cuối mảng**. Trả về độ dài mới của mảng.
- Cú pháp: **array.push(item1, item2, ..., itemX)**
- Ví dụ:

```
var list = ["HTML5", "CSS3", "Javascript"];  
console.log(list.push("Bootstrap 4", "ReactJS")); // Trả về: 5  
console.log(list);  
// Trả về: ["HTML5", "CSS3", "Javascript", "Bootstrap 4", "ReactJS"]
```

09. Làm việc với Array

9.5. shift()

- Dùng để xóa phần tử đầu mảng và trả về phần tử ở đầu mảng đó.
- Cú pháp: `array.shift()`
- Ví dụ:

```
var list = ["HTML5", "CSS3", "Javascript"];
console.log(list.shift()); // Trả về: "HTML5"
console.log(list); // Trả về: ["CSS3", "Javascript"]
```

09. Làm việc với Array

9.6. unshift()

- **Thêm** một hoặc nhiều **phần tử vào đầu mảng**. Trả về độ dài mới của mảng.
- Cú pháp: **array.unshift(item1, item2, ..., itemX)**
- Ví dụ:

```
var list = ["HTML5", "CSS3", "Javascript"];  
console.log(list.unshift("Bootstrap 4", "ReactJS")); // Trả về: 5  
console.log(list);  
// Trả về: ["Bootstrap 4", "ReactJS", "HTML5", "CSS3", "Javascript"]
```

09. Làm việc với Array

9.7. splice()

- **Xóa hoặc chèn phần tử** mới vào mảng. Trả về mảng bị xóa.
- Cú pháp: `array.splice(index, howmany, item1, ....., itemX)`
- Trong đó:
 - **index**: Vị trí thêm/xóa phần tử (bắt buộc).
 - **howmany**: Số phần tử cần xóa (không bắt buộc).
 - **item1, ..., itemX**: Các phần tử mới được thêm vào (không bắt buộc).

- Ví dụ 1: Chèn phần tử mới vào mảng

```
var list = ["HTML5", "CSS3", "Javascript"];  
console.log(list.splice(2, 0, "Bootstrap 4", "ReactJS")); // Trả về: []  
console.log(list);  
// Trả về: ["HTML5", "CSS3", "Bootstrap 4", "ReactJS", "Javascript"]
```


09. Làm việc với Array

9.7. splice()

- Ví dụ 2: Xóa phần tử trong mảng

```
var list = ["HTML5", "CSS3", "Javascript"];  
console.log(list.splice(1, 1)); // Trả về: ["CSS3"]  
console.log(list); // Trả về: ["HTML5", "Javascript"]
```

- Ví dụ 3: Xóa phần tử và chèn phần tử mới vào mảng

```
var list = ["HTML5", "CSS3", "Javascript"];  
console.log(list.splice(2, 1, "Bootstrap 4", "ReactJS"));  
// Trả về: ["Javascript"]  
console.log(list);  
// Trả về: ["HTML5", "CSS3", "Bootstrap 4", "ReactJS"]
```

09. Làm việc với Array

9.8. concat()

- Dùng để **nối 2 array**. Không làm ảnh hưởng đến mảng ban đầu.

- Cú pháp: **array1.concat(array2, array3, ..., arrayX)**

- Ví dụ:

```
var list = ["HTML5", "CSS3", "Javascript"];
var list2 = ["Bootstrap 4", "ReactJS"];
console.log(list.concat(list2));
// Trả về: ["HTML5", "CSS3", "Javascript", "Bootstrap 4", "ReactJS"]
console.log(list);
// Trả về: ["HTML5", "CSS3", "Javascript"]
```

09. Làm việc với Array

9.9. slice()

- Dùng để **cắt các phần tử**. Không làm ảnh hưởng đến mảng ban đầu.
- Cú pháp: **array.slice(start, end)**
- Trong đó:
 - **start**: Vị trí bắt đầu. Mặc định là 0. (không bắt buộc).
 - **end**: Vị trí kết thúc. Mặc định là phần tử cuối cùng. (không bắt buộc).

- Ví dụ 1: Chèn phần tử mới vào mảng

```
var list = ["HTML5", "CSS3", "Javascript", "Bootstrap 4", "ReactJS"];  
console.log(list.slice(3)); // Trả về: ["Bootstrap 4", "ReactJS"]  
console.log(list.slice(1, 3)); // Trả về: ["CSS3", "Javascript"]  
console.log(list.slice(-3, -1)); // Trả về: ["Javascript", "Bootstrap 4"]  
console.log(list);  
// Trả về: ["HTML5", "CSS3", "Javascript", "Bootstrap 4", "ReactJS"]
```



FRONT-END

Buổi 7: Javascript cơ bản (Tiết 2)



Nội dung

01

Câu lệnh rẽ nhánh If Else

02

Switch Case

03

Vòng lặp For

04

Vòng lặp For In

05

Vòng lặp For Of

06

Vòng lặp While

07

Break

08

Continue

09

Variables (Biến)

01. Câu lệnh rẽ nhánh If Else

1.1. Câu lệnh if

- Câu lệnh if(...) sẽ **kiểm tra điều kiện** biểu thức bên **trong cặp dấu ngoặc đơn ()**. Nếu kết quả là **true** thì một khối code sẽ **được thực thi**.
- Cú pháp:

```
if (condition) {  
    //nếu điều kiện đúng thì thực hiện  
}
```
- Trong đó **condition** (điều kiện) là mệnh đề điều kiện và luôn luôn phải có một trong hai giá trị là true/false hoặc giá

01. Câu lệnh rẽ nhánh If Else

1.2. Lệnh if else

- Lệnh **else** sẽ được thực thi nếu lệnh if không được thực hiện, tức là điều kiện ở **condition** sẽ có giá trị là **false**.
- **Lưu ý:** khi dùng lệnh else thì bắt buộc **phải có** một lệnh if đứng **trước nó**.

- Cú pháp:

```
if (condition) {
```

```
    // Code cho lệnh if
```

```
}
```

```
else {
```

01. Câu lệnh rẽ nhánh If Else

1.3. Kết hợp nhiều lệnh if else

- Ta có thể kết hợp nhiều câu lệnh if để xử lý bài toán, bằng cách thêm nhiều lệnh **else if** phía sau.
- Cú pháp:

```
if (condition1) {  
    // Code cho lệnh if 1  
}  
  
else if (condition2) {  
    // Code cho lệnh if 2  
}
```


01. Câu lệnh rẽ nhánh If Else

1.4. Lệnh if else lồng nhau

- Ví dụ: (thử các trường hợp gán giá trị cho a, b)

```
var a;  
var b;  
if (a !== undefined && b !== undefined) {  
    if (a === b) {  
        console.log("a bằng b");  
    } else {  
        console.log("a khác b");  
    }  
} else {  
    console.log("a hoặc b chưa được gán giá trị.");  
}
```

01. Câu lệnh rẽ nhánh If Else

1.5. Câu lệnh if else rút gọn (Toán tử 3 ngôi - Ternary operator)

- Chúng ta cũng có thể **rút gọn** câu lệnh **if else** bằng cách dùng toán tử rẽ nhánh ? (hoặc có thể gọi là **toán tử dấu hỏi**).

- Cú pháp:

var result = condition ? value1 : value2;

- Đây là toán tử **ba ngôi** với ba thành phần:
 - **condition**: điều kiện cần kiểm tra
 - **value1**: trả về giá trị **value1** nếu điều kiện **condition** là **true**.
 - **value2**: trả về giá trị **value2** nếu điều kiện **condition** là **false**.

02. Switch Case

- Lệnh **switch case** có công dụng giống như lệnh if else đó là để **dùng để rẽ nhánh** chương trình.
- Ứng với **mỗi nhánh sẽ có một điều kiện** cụ thể và **điều kiện** đó phải sử dụng toán tử **so sánh bằng**.
- Còn đối với lệnh if else thì bạn có thể **truyền vào một mệnh đề bất kì** và sử dụng nhiều toán tử khác nhau.

- Cú pháp:

```
switch (variable) {  
    case value_1:  
        // do some thing  
        break;  
    case value_2:  
        // do some thing  
        break;  
    default:  
        // do something  
        break;  
}
```

02. Switch Case

- Trường hợp **bỏ break** đi thì câu lệnh sẽ **chạy tiếp xuống case tiếp theo**, đến case nào có **break** thì **dừng lại**.
- Trường hợp **không có default** thì nếu **rơi vào trường hợp này sẽ không in ra gì**.
- Ta cũng có thể **gom nhóm case** lại.

03. Vòng lặp For

- Vòng lặp **for** dùng để **lặp lại** việc thực thi **một đoạn mã** nào đó **một số lần**.

- Cú pháp:

```
for (bienkhoitao; dieukienthucthi; buocnhay) {  
    // code...  
}
```

- Trong đó:

- **bienkhoitao**: là **giá trị khởi tạo** ban đầu của vòng lặp.
- **dieukienthucthi**: là **điều kiện** mà vòng lặp được phép chạy (**chú ý**: Nếu bạn bỏ trống sẽ bị lặp vô tận).

04. Vòng lặp For In

- Vòng lặp **for in** dùng để **lấy ra key** của một **object** (cũng có thể áp dụng cho **array, string**).

- Cú pháp:

```
for (key in object) {  
    //Viết code ở đây  
}
```

- Trong đó:

- **key**: là tên biến (*đặt tên là gì cũng được*).
- **object**: là một object, **mỗi key** sẽ tương ứng với **key ở trong object**.

05. Vòng lặp For Of

- Vòng lặp **for of** dùng để lấy ra phần tử của một **array**, **string** (không áp dụng được với **object**).

- Cú pháp:

```
for (variable of iterable) {  
    //Viết code ở đây  
}
```

- Trong đó:

- **variable**: là tên biến, lấy ra các giá trị trong mảng.
- **iterable**: là một mảng hoặc **string**, mỗi **variable** sẽ tương ứng với từng giá trị ở trong mảng hoặc string.

o6. Vòng lặp While

6.1. Vòng lặp while

- Vòng lặp **while** luôn luôn kiểm tra điều kiện trước, nếu điều kiện **thỏa mãn** thì **mới chạy** vào trong vòng lặp.
- Cú pháp:

```
while (condition) {  
    // code  
}
```
- Trong đó:
 - **condition**: là **điều kiện** thực hiện vòng lặp.

o6. Vòng lặp While

6.2. Vòng lặp do...while

- Khác với vòng lặp while, vòng lặp **do...while** **luôn thực hiện ít nhất một lượt lặp**. Sau đó mới kiểm tra điều kiện lặp.
- Cú pháp:
do {
 // code
} while (condition);
- Trong đó:
 - **condition**: là điều kiện thực hiện vòng lặp.

07. Break

- Lệnh **break** có tác dụng **dừng vòng lặp** cho dù điều kiện của vòng lặp **vẫn đang đúng**.

o8. Continue

- Lệnh **continue** có tác dụng **bỏ qua một bước lặp** nào đó.
- Nghĩa là lúc **gặp lệnh continue** thì tất cả những đoạn code nằm bên dưới sẽ không được thực hiện mà nó sẽ **nhảy sang vòng lặp mới** luôn.

09. Variables (Biến)

9.1. Khái niệm

- **Biến** (variable) là nơi để bạn có thể **lưu trữ thông tin**.

09. Variables (Biến)

9.2. Cách đặt tên cho biến

- **Tên biến** phải được đặt theo **quy tắc** sau đây:
 - Tên biến bắt đầu phải là chữ cái hoặc ký tự '_'
 - Tên biến **không** được **bắt đầu bằng số**.
 - Tên biến **không** được **bắt đầu bằng ký tự đặc biệt**.
- Đặt tên biến **ĐÚNG**:
 - `var tenbien = 10;`
 - `var _tenbien = 10;`
 - `var TenBien; = 10;`
 - `var ten_bien = 10; // snake_case`
 - `var tenBien = 10; // camelCase`
 - `var tenbien123 = 10;`
- Đặt tên biến **SAI**:
 - `var 123tenbien = 10; // Do số đứng đầu`
 - `var -tenbien = 10; // Do có ký tự -`

09. Variables (Biến)

9.3. Các cách khai báo biến

- Khai báo biến bằng **từ khóa var** (variable - biến)
 - Cú pháp: **var** tenBien = giaTri;
 - Trong đó:
 - tenBien: Là tên của biến các bạn muốn đặt.
 - giaTri: Là giá trị của biến, có thể là số, chuỗi, mảng, object.
 - Lưu ý: Tên biến có phân biệt chữ hoa, chữ thường.

09. Variables (Biến)

9.3. Các cách khai báo biến

- Khai báo biến bằng **từ khóa let**
- Cú pháp: **let** tenBien = giaTri;
 - Biến **let** chỉ có **phạm vi trong khối khai báo** mà thôi.
 - Vì vậy nó sẽ tối ưu trong trường hợp bạn cần khai báo một **biến chỉ sử dụng tạm thời trong một phạm vi nhất định**. Phạm vi này ta hay gọi là block scope.

09. Variables (Biến)

9.3. Các cách khai báo biến

- Khai báo biến bằng **từ khóa const**
- Cú pháp: **const** tenBien = giaTri;
 - **const** là một **hằng số**, vì vậy khi khai báo biến const thì bạn phải gán giá trị cho nó luôn, sau này cũng **không thể thay đổi giá trị** cho biến.

09. Variables (Biến)

9.3. Các cách khai báo biến

- Khai báo biến **không cần dùng từ khóa**
- Cú pháp: `tenBien = giaTri;`
 - Chỉ nên sử dụng cách này khi ta **muốn gán lại giá trị của một biến.**



FRONT-END

Buổi 8: Javascript cơ bản (Tiết 3)



Nội dung

01

Functions

02

Try Catch

03

Làm việc với Object nâng cao

04

Làm việc với Array nâng cao

01. Functions

1.1. Kiến thức tổng quan

- Hàm là một **chương trình con** giúp **thực thi một công việc cụ thể**.
- Có **2 loại** hàm:
 - Hàm **built-in** (Hàm có sẵn trong Javascript): `console.log()`, `alert()`, `prompt()`, ...
 - Hàm **tự định nghĩa**: Là hàm do **chúng ta tự viết** ra. Ví dụ như hàm tính toán cộng, trừ, nhân, chia.
- Cú pháp hàm tự định nghĩa:

```
function tenHam(thamSo1, thamSo2,...) {
```

01. Functions

1.1. Kiến thức tổng quan

- Đối tượng **arguments**: chỉ dùng được ở trong hàm. Giả sử khi gọi hàm và **truyền đối số** vào thì **tất cả các đối số** đó sẽ được **nằm trong arguments** dưới dạng **một mảng**.
- Từ khóa **return**: sẽ **trả về kết quả** nằm ở **bên phải** của chữ **return**. Những dòng **code** ở **bên dưới return** sẽ **không hoạt động**.
- Lưu ý:

Nếu đặt trùng tên hàm khi ta gọi hàm thì việc gọi hàm này sẽ

01. Functions

1.2. Các loại hàm (các cách viết hàm)

1.2.1. Declaration function (Hàm định nghĩa)

- Có tính **hoisting** (tức là gọi được hàm trước lúc khai báo hàm đó).
- Cú pháp:

```
function tenHam(thamSo1, thamSo2,...) {  
  
    // Code  
  
}
```

01. Functions

1.2. Các loại hàm (các cách viết hàm)

1.2.2. Expression function (Hàm biểu thức)

- **Không** có tính **hoisting** (Nếu gọi hàm trước lúc khai báo hàm thì sẽ gặp lỗi).
- Cú pháp:

```
var tenHam = function(thamSo1, thamSo2,...) {  
  
    // Code  
  
}
```

01. Functions

1.2. Các loại hàm (các cách viết hàm)

1.2.3. Arrow function (Hàm mũi tên)

- **Không** có tính **hoisting** (Nếu gọi hàm trước lúc khai báo hàm thì sẽ gặp lỗi).
- Cú pháp:

```
var tenHam = (thamSo1, thamSo2,...) => {  
    // Code  
}
```

- Lưu ý: **Arrow Function** sẽ **không** có **Arguments**. Để lấy các đối số đã truyền dưới dạng một mảng thì chúng ta sẽ sử dụng `arguments`.

01. Functions

1.3. Ví dụ về hàm

- **Ví dụ 1:** Tính tổng các số chẵn của mảng. Mảng như sau:

`var mang = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];`

- **Ví dụ 2:**Viết chương trình cho người dùng nhập vào số n lớn hơn 0, sau đó dựa vào n tính giá trị của biểu thức:

$$\text{bieuThuc} = 1/n + 2/n + 3/n + \dots + n/n.$$

02. Try Catch

- **Try catch** là một khối lệnh dùng để bắt lỗi chương trình.
- Ta sử dụng **try catch** khi muốn **chương trình không bị dừng** khi một **lệnh nào đó bị lỗi**. Thường thì đó là những lỗi do người dùng nhập sai dữ liệu, hoặc người dùng thao tác bị sai.
- Cú pháp:

```
try {  
    // Chạy vào đây đầu tiên  
} catch (error) {  
    // Nếu lỗi chạy vào đây  
} finally {  
    // Luôn luôn chạy vào đây và chạy sau cùng  
}
```

02. Try Catch

- **Các bước thực thi:**

- **Bước 1:** Thực hiện trong try.
- **Bước 2:** Nếu trong try xuất hiện lỗi thì các câu lệnh trong try không được thực thi. Và sẽ nhảy sang catch, các câu lệnh trong catch sẽ được thực thi.
- **Bước 3:** Cuối cùng nhảy xuống finally dù là lỗi hay không. finally là lựa chọn thêm, không bắt buộc phải thêm vào.

- **Lưu ý:** Nếu dùng try catch thì nếu trong try catch gặp lỗi thì các câu lệnh bên ngoài try catch vẫn chạy bình thường. Nếu không dùng try catch mà gặp phải lỗi thì các câu lệnh phía sau sẽ bị dừng.

03. Làm việc với Object nâng cao

3.1. Cách thêm 1 key mới vào object

- Cú pháp:

```
tenObject.tenKey = value;
```

- Hoặc:

```
tenObject["tenKey"] = value;
```

3.2. Cách xóa 1 key khỏi object

- Cú pháp:

```
delete tenObject.tenKey;
```

04. Làm việc với Array nâng cao

4.1. forEach()

- Để duyệt qua mỗi phần tử của mảng và thực hiện một hành động nào đó.
- Có thể thay đổi trực tiếp mảng ban đầu
- Cú pháp:

```
arr.forEach(function(currentValue, index, array) {  
    // code xử lý  
});
```

- Trong đó:
 - **currentValue**: phần tử hiện tại đang được xử lý của array.
 - **index**: chỉ số của phần tử hiện tại đang được xử lý của array.
 - **array**: mảng hiện tại đang gọi hàm forEach().

[ → ) → [

04. Làm việc với Array nâng cao

4.2. every()

- Kiểm tra tất cả các phần tử của một mảng phải thỏa mãn một điều kiện gì đó.
- Cú pháp:

```
arr.every(function(currentValue, index, array) {  
    // code xử lý  
});
```

- Trong đó:
 - **currentValue**: phần tử hiện tại đang được xử lý của array.
 - **index**: chỉ số của phần tử hiện tại đang được xử lý của array.
 - **array**: mảng hiện tại đang gọi hàm every().

.every( === ) → false

04. Làm việc với Array nâng cao

4.3. some()

- Kiểm tra chỉ cần một phần tử của một mảng thỏa mãn một điều kiện gì đó là được.
- Cú pháp:

```
arr.some(function(currentValue, index, array) {  
    // code xử lý  
});
```

- Trong đó:
 - **currentValue**: phần tử hiện tại đang được xử lý của array.
 - **index**: chỉ số của phần tử hiện tại đang được xử lý của array.
 - **array**: mảng hiện tại đang gọi hàm some().

.some( === ) → true

04. Làm việc với Array nâng cao

4.4. find()

- Tìm xem trong mảng có giá trị giống với giá trị đang cần tìm không, nếu có thì trả về chính phần tử của mảng đó. Nếu không có thì trả về undefined.
- Chỉ tìm được 1 phần tử trong mảng.
- Cú pháp:

```
arr.find(function(currentValue, index, array) {  
    // code xử lý  
});
```

- Trong đó:
 - **currentValue**: phần tử hiện tại đang được xử lý của array.
 - **index**: chỉ số của phần tử hiện tại đang được xử lý của array.
 - **array**: mảng hiện tại đang gọi hàm find().



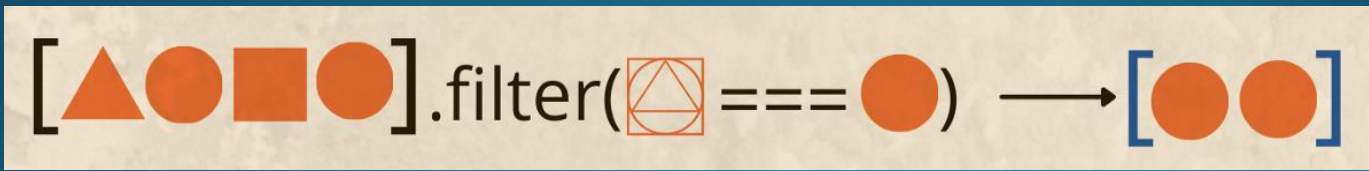
04. Làm việc với Array nâng cao

4.5. filter()

- Giống find, nhưng cho phép tìm được nhiều phần tử và trả về một array gồm các phần tử đã tìm được.
- Cú pháp:

```
arr.filter(function(currentValue, index, array) {  
    // code xử lý  
});
```

- Trong đó:
 - **currentValue**: phần tử hiện tại đang được xử lý của array.
 - **index**: chỉ số của phần tử hiện tại đang được xử lý của array.
 - **array**: mảng hiện tại đang gọi hàm filter().



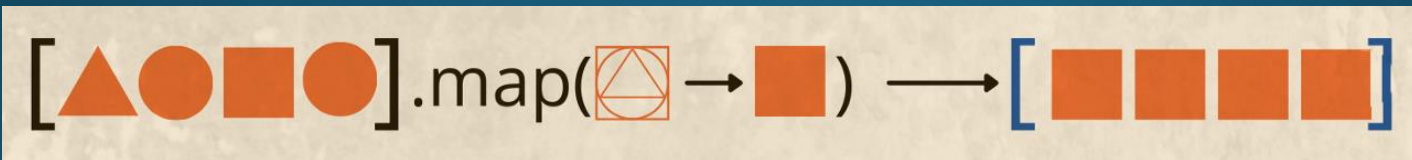
04. Làm việc với Array nâng cao

4.6. map()

- Hàm map() sẽ lặp qua từng phần tử của mảng. Giá trị trả về của hàm map là một mảng mới, với số lượng phần tử bằng với mảng cũ, nhưng giá trị của các phần tử thì được quyết định bởi lệnh return của hàm map.
- Cú pháp:

```
arr.map(function(currentValue, index, array) {  
    // code xử lý  
});
```

- Trong đó:
 - **currentValue**: phần tử hiện tại đang được xử lý của array.
 - **index**: chỉ số của phần tử hiện tại đang được xử lý của array.
 - **array**: mảng hiện tại đang gọi hàm map().



04. Làm việc với Array nâng cao

4.7. reduce()

- Hàm `reduce()` sẽ duyệt qua từng phần tử trong mảng và tính toán các phần tử đó, sau đó trả về một giá trị cuối cùng.

- Cú pháp:

```
arr.reduce(function(total, currentValue, currentIndex, array) {  
    // code xử lý  
}, initialValue);
```

- Trong đó:

- **total**: giá trị trả lại trước đó của function, chính là giá trị của lệnh `return` cho mỗi lần lặp.
- **initialValue**: tham số không bắt buộc. Nếu được truyền vào thì `initialValue` sẽ được sử dụng làm giá trị ban đầu, còn không thì nó sẽ lấy giá trị của phần tử đầu tiên.

